

## Skill Week 5

### Task 1

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
GNU nano 7.2 navigation.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

int main() {
    char path[PATH_MAX];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";

    printf("Simple Directory Navigation Program\n");
    printf("Commands: cd <path>, pwd, previous, exit\n\n");

    while (1) {
        // Display current directory
        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            break;
        }

        printf("%s$ ", current_dir);

        if (fgets(path, sizeof(path), stdin) == NULL) {
            break;
        }
    }
}
```

```

    }
    path[strcspn(path, "\n")] = '\0';
    if (strcmp(path, "exit") == 0) {
        printf("Exiting...\n");
        break;
    }
    if (strcmp(path, "pwd") == 0) {
        printf("Current Directory: %s\n", current_dir);
        continue;
    }
    if (strcmp(path, "previous") == 0) {
        if (strlen(previous_dir) == 0) {
            printf("No previous directory available.\n");
            continue;
        }
        char temp[PATH_MAX];
        strcpy(temp, current_dir);
        if (chdir(previous_dir) == -1) {
            perror("chdir");
            continue;
        }
        strcpy(previous_dir, temp);
        printf("Changed to previous directory.\n");
        continue;
    }
    if (strncmp(path, "cd ", 3) == 0) {
        char *new_path = path + 3;
        while (*new_path == ' ') {
            new_path++;

```

```

        if (stat(new_path, &st) == -1) {
            perror("Invalid path");
            continue;
        }
        if (!S_ISDIR(st.st_mode)) {
            printf("Error: Path is not a directory.\n");
            continue;
        }
        strcpy(previous_dir, current_dir);
        if (chdir(new_path) == -1) {
            perror("chdir");
            continue;
        }

        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            continue;
        }
        printf("Directory changed successfully.\n");
        printf("New Working Directory: %s\n", current_dir);
    }
    else {
        printf("Invalid command.\n");
        printf("Use: cd <path>, pwd, previous, or exit\n");
    }
}
return 0;
}

```

OUTPUT:

```
root@pranay:~# nano navigation.c
root@pranay:~# gcc navigation.c -o navigation
root@pranay:~# ./navigation
Simple Directory Navigation Program
Commands: cd <path>, pwd, previous, exit

/root$ pwd
Current Directory: /root
```

## Task 2

To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 100
#define MAX_ARGS 20
/* Function declarations */
void command_pwd(char **args);
void command_cd(char **args);
void command_help(char **args);
void command_exit(char **args);
/* Dispatch table structure */
typedef struct {
    char *name;
    void (*function)(char **args);
} Command;
/* Built-in command table */
Command dispatch_table[] = {
    {"pwd", command_pwd},
    {"cd", command_cd},
    {"help", command_help},
    {"exit", command_exit}
};
int command_count = sizeof(dispatch_table) / sizeof(dispatch_table[0]);
/* pwd command */
void command_pwd(char **args) {
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
```

```

/* cd command */
void command_cd(char **args) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return;
    }
    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

/* help command */
void command_help(char **args) {
    printf("\nAvailable commands:\n");
    printf("  pwd      - Show current directory\n");
    printf("  cd <dir> - Change directory\n");
    printf("  help     - Show available commands\n");
    printf("  exit     - Exit shell\n");
    printf("  Other commands are executed as processes.\n\n");
}

/* exit command */
void command_exit(char **args) {
    printf("Exiting shell...\n");
    exit(0);
}

/* Execute external process */
void execute_process(char **args) {
    pid_t pid = fork();
    if (pid < 0) {

```

```

        if (execvp(args[0], args) == -1) {
            perror("Command execution failed");
            exit(1);
        }
    } else {
        /* Parent process */
        waitpid(pid, NULL, 0);
    }
}

/* Check dispatch table */
int execute_builtin(char **args) {
    for (int i = 0; i < command_count; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            dispatch_table[i].function(args);
            return 1;
        }
    }
    return 0;
}

/* Convert input into arguments */
int parse_input(char *input, char **args) {
    int count = 0;
    char *token = strtok(input, " \t\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[count] = NULL;
    return count;
}

```

```
char *args[MAX_ARGS];
printf("=====\n");
printf("        SIMPLE COMMAND SHELL\n");
printf("=====\n");
while (1) {
    printf("myshell> ");
    fflush(stdout);
    /* Read command */
    if (fgets(input, sizeof(input), stdin) == NULL) {
        break;
    }
    /* Ignore empty input */
    if (strlen(input) <= 1) {
        continue;
    }
    /* Convert input into arguments */
    int argc = parse_input(input, args);
    if (argc == 0) {
        continue;
    }
    /* First check built-in commands */
    if (execute_builtin(args)) {
        continue;
    }
}
return 0;
}
```

</

```
root@pranay:~# nano Shell.c
root@pranay:~# gcc Shell.c -o Shell
root@pranay:~# ./Shell
=====
                SIMPLE COMMAND SHELL
=====
myshell> pwd
/root
myshell> ls
myshell> help

Available commands:
  pwd          - Show current directory
  cd <dir>     - Change directory
  help         - Show available commands
  exit         - Exit shell
  Other commands are executed as processes.

myshell> exit
Exiting shell...
root@pranay:~# |
```

## Task

3:

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>
void cleanup() {
    printf("\nCleaning up resources...\n");
    printf("Resources cleaned successfully.\n");
}
int main() {
    char cwd[PATH_MAX];
    /* Register cleanup function */
    atexit(cleanup);
    printf("=== Directory and Process Management ===\n\n");
    /* Retrieve current directory */
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("Current Directory: %s\n", cwd);
    } else {
        perror("getcwd");
        return 1;
    }
    /* Display path */
    printf("Path: %s\n", cwd);
    /* Save state */
    FILE *file = fopen("state.txt", "w");
    if (file != NULL) {
        fprintf(file, "Saved Working Directory: %s\n", cwd);
        fclose(file);
        printf("State saved successfully.\n");
    }
}

```

```

printf("Path: %s\n", cwd);
/* Save state */
FILE *file = fopen("state.txt", "w");
if (file != NULL) {
    fprintf(file, "Saved Working Directory: %s\n", cwd);
    fclose(file);
    printf("State saved successfully.\n");
} else {
    perror("State file");
}
/* Verify correct behavior */
if (getcwd(cwd, sizeof(cwd)) != NULL) {
    printf("Verification: Current directory is correct.\n");
} else {
    printf("Verification failed.\n");
}
/* Process exit request */
char choice;
printf("\nDo you want to exit? (y/n): ");
scanf(" %c", &choice);
if (choice == 'y' || choice == 'Y') {
    printf("Process exit requested.\n");
    exit(0);
} else {
    printf("Process will continue running.\n");
}
return 0;
}

```

OUTPUT:

```

root@balaji@BalajiPatil:~# nano directory.c
root@balaji@BalajiPatil:~# gcc directory.c -o directory
root@balaji@BalajiPatil:~# ./directory
=== Directory and Process Management ===

Current Directory: /root
Path: /root
State saved successfully.
Verification: Current directory is correct.

```

```
Do you want to exit? (y/n): y
Process exit requested.

Cleaning up resources...
Resources cleaned successfully.
```

## Task 4

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 200
#define MAX_NAME 100
#define MAX_VALUE 100
/* Validate environment variable name */
int valid_name(char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;
    /* First character must be letter or underscore */
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;
    /* Remaining characters */
    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }
    return 1;
}
/* Process export command */
void export_variable(char *input)
{
    char name[MAX_NAME];
```

```

char *equal = strchr(assignment, '=');
if (equal == NULL)
{
    printf("Error: Invalid export syntax.\n");
    printf("Usage: export NAME=VALUE\n");
    return;
}
/* Extract variable name */
int name_length = equal - assignment;
if (name_length <= 0 || name_length >= MAX_NAME)
{
    printf("Error: Invalid variable name.\n");
    return;
}
strncpy(name, assignment, name_length);
name[name_length] = '\0';
/* Extract value */
strcpy(value, equal + 1);
/* Validate variable name */
if (!valid_name(name))
{
    printf("Error: Invalid variable name: %s\n", name);
    return;
}
/* Check whether variable already exists */
char *old_value = getenv(name);
if (old_value != NULL)
{
    printf("Existing variable found: %s=%s\n", name, old_value);
}

```

```

{
    perror("setenv");
}
}
/* Test that variable is exported to child process */
void test_child_process()
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        return;
    }
    if (pid == 0)
    {
        printf("\nChild Process:\n");
        char *value = getenv("MYVAR");
        if (value != NULL)
            printf("MYVAR=%s\n", value);
        else
            printf("MYVAR is not set.\n");
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("Child process completed.\n");
    }
}

```

```

printf("          EXPORT COMMAND TESTER\n");
printf("=====\\n");
while (1)
{
    printf("\\nshell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        break;
    /* Remove newline */
    input[strcspn(input, "\\n")] = '\\0';
    /* Exit */
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting...\\n");
        break;
    }
    /* Export command */
    if (strncmp(input, "export ", 7) == 0)
    {
        export_variable(input);
    }
    /* Test child process */
    else if (strcmp(input, "test") == 0)
    {
        test_child_process();
    }
    /* Display variable */
    else if (strncmp(input, "print ", 6) == 0)
    {
        char *name = input + 6;

```

```

}
/* Display variable */
else if (strncmp(input, "print ", 6) == 0)
{
    char *name = input + 6;
    char *value = getenv(name);
    if (value != NULL)
        printf("%s=%s\\n", name, value);
    else
        printf("%s is not set.\\n", name);
}
else
{
    printf("Unknown command.\\n");
}
}
return 0;
}

```

OUTPUT:

```
root@balaji@BalajiPatil:~# nano export.c
root@balaji@BalajiPatil:~# gcc export.c -o export
root@balaji@BalajiPatil:~# ./export
=====
EXPORT COMMAND TESTER
=====

shell> export MYVAR=Hello
Export successful: MYVAR=Hello

shell> get MYVAR
Unknown command.

shell> test

Child Process:
MYVAR=Hello
Child process completed.

shell> exit
Exiting...
root@balaji@BalajiPatil:~# |
```